

Abstract

The **Monkey Head Project (MHP)** is an experimental robotics platform integrating advanced AI through a custom AI Operating System (*GenCore*) and a high-performance hardware suite. This paper presents the full technical specifications of MHP's hardware – including a **Lenovo Legion Go** handheld (AMD Ryzen Z1 Extreme, 16 GB LPDDR5x RAM, 512 GB NVMe SSD, 8.8" 2560×1600 @ 144 Hz display ¹ ²) and clusters of **Raspberry Pi 4** boards (Broadcom BCM2711 SoC, quad Cortex-A72 1.5–1.8 GHz, up to 8 GB RAM ³ ⁴) – alongside the modular **Huey** robotic shell built with engineering-grade components (Supermicro X9QRI-F+ server-class motherboard, ECC RAM, Intel Xeon CPUs, NVMe storage, liquid cooling) ⁵. The software stack is based on **Debian 13 "Trixie"** Linux and Python 3.12, incorporating modern AI frameworks and libraries (e.g. *LangChain*, *LlamaIndex*, OpenAI API) to leverage large language models. A detailed account is given of MHP's AI applications: notably, the integration of **OpenAI Codex** – an AI model that translates natural language to code ⁶ – to automate code generation, scripting, and real-time logical adaptations within the GenCore OS. We outline the distributed system architecture spanning mobile, embedded, and desktop environments: a three-tier AIOS (HostOS for strategic control, SubOS for task orchestration, NanoOS for device-level real-time control ⁷) interconnected across platforms via containerization and APIs. Practical use-cases are discussed, focusing on MHP's deployment as a portable "**Huey**" lab assistant for field science and engineering work. We present comparative performance analyses of AI-driven data processing using **plaintext (TXT)** vs **JSON** file formats, evaluating storage efficiency, parse speed, human-readability, and integration overhead. Key challenges – including firmware compatibility when interfacing legacy computers, battery life constraints on mobile units (≤ 3 hours on Legion Go under load ²), thermal management for high-TDP components (addressed with custom liquid cooling ⁵ and even water-cooled Raspberry Pi clusters ⁸), and multi-platform software compatibility – are examined in depth. Finally, we describe optimizations and outline future plans for scaling up the platform (e.g. larger compute clusters, improved energy autonomy ⁹) and increasing automation in both AI reasoning and robotic operation. The findings underscore MHP's potential as a modular, AI-driven robotics framework and highlight lessons learned at the intersection of modern AI and hardware systems.

Introduction

The **Monkey Head Project** is a multidisciplinary initiative to create a *universal AI/robotics platform* that marries cutting-edge artificial intelligence with diverse hardware systems ¹⁰. At its heart is **GenCore**, an adaptive AI Operating System designed to coordinate actions across a network of computing devices and robotic components. The project's core vision is to **democratize robotics and AI**, making advanced autonomous systems accessible and ethically governed ¹⁰. In essence, MHP posits that a single determined individual, equipped with modern AI tools, can design a fully autonomous robot spanning from legacy 1980s computers to contemporary gaming handhelds and servers ¹¹.

OpenAI Codex integration is a distinguishing feature of MHP, enabling AI-driven code generation and system self-modification. OpenAI Codex is an AI model (derived from GPT-3) specialized for programming tasks, capable of translating natural language instructions into code in over a dozen languages ⁶ ¹². By leveraging Codex (and its derivative tools such as GitHub Copilot), MHP's GenCore can automate writing of

new scripts or algorithms on the fly. This allows the system to adapt in real time – for example, generating Python code to interface with a new sensor or to analyze experimental data – without human intervention. Such *AI-assisted coding* is intended to accelerate development and enable on-site problem solving, as Codex can produce solutions to well-defined tasks (OpenAI reports Codex could correctly fulfill ~37% of code requests in one pass ¹³) or at least provide a strong starting point for human refinement.

This paper provides an academic-style exposition of the Monkey Head Project’s design and capabilities. We begin by detailing the **hardware and software specifications** of the platform, encompassing the custom “Huey” robotic shell and the various computing units in use. Next, we explain the **AI applications and integrations** within MHP, with emphasis on how Codex and related AI models support automation, decision-making, and cross-platform operation. We then describe the **system architecture** – a modular, layered OS deployed across mobile (e.g. Legion Go), embedded (Raspberry Pi cluster), and desktop/server hardware (Huey’s Supermicro-based system) – and illustrate how these components interoperate as a cohesive whole. In the **Methods** section, we also outline how the software was developed and configured (e.g. use of Debian Trixie environments, containerization via Docker/Kubernetes). The **Results** section presents a comparison of data handling performance using plain text versus JSON formats, which arose from experiments in optimizing MHP’s data logging and knowledge storage. In the **Discussion**, we examine the practical use of MHP as a field-deployable lab assistant – nicknamed “Huey” – and candidly address the **challenges and limitations** encountered during development (such as firmware and compatibility issues when interfacing with legacy systems, battery and thermal constraints of running intensive AI on portable devices, etc.). We also discuss the strategies employed to mitigate these issues (from hardware modifications to software workarounds). Finally, we consider **future work** and optimizations planned for MHP, including scaling up the system’s computational power, improving its autonomy and efficiency, and enhancing the AI’s reliability and scope of capabilities.

By combining documentation from the project’s repository and relevant technical literature, this paper aims to serve as a comprehensive case study of integrating state-of-the-art AI (OpenAI Codex and beyond) with heterogeneous robotic hardware. The broader goal is to inform and inspire similar efforts in **AI-assisted robotics**, highlighting what is possible today and what hurdles remain on the path toward fully autonomous, intelligent systems.

Methods

Hardware Specifications and Configuration

MHP Hardware Overview: The Monkey Head Project utilizes a heterogeneous collection of hardware, from modern PCs to single-board microcomputers and legacy machines, each chosen to fulfill specific roles in the system. Table 1 summarizes the key hardware components and their technical specifications, followed by details of how these components are configured within the MHP architecture.

- **Lenovo Legion Go (Handheld PC):** Used as a development and mobile computing platform, the Legion Go is a handheld gaming computer with high-end specs for its size. It features an AMD Ryzen Z1 Extreme 8-core APU (Zen 4 CPU + RDNA 3 GPU) running up to ~3.3 GHz, **16 GB LPDDR5X** RAM, and a **512 GB PCIe 4.0 NVMe SSD** ¹⁴ ¹⁵. The device has an 8.8-inch QHD+ (2560×1600) IPS touchscreen capable of 144 Hz refresh ¹⁶. For connectivity, it offers Wi-Fi 6E and Bluetooth 5.3, dual USB-C (USB4/DisplayPort Alt Mode) ports, and a 3.5 mm audio jack ¹⁷. Running Windows 11 by

default ¹⁸, the Legion Go in MHP has been configured to also boot into **Debian Trixie Linux** (via a dual-boot or live environment) to maintain consistency with the project's Linux-based stack. This device serves as a **mobile HostOS unit** for demonstrations – its combination of a powerful CPU/GPU and built-in display/input make it ideal for showcasing GenCore's capabilities on the go. A major constraint is its **battery**: a 49.2 Wh lithium-ion battery provides only **~2-3 hours of runtime under heavy AI workloads** (e.g. when using the GPU for inference), even after power mode tweaks ². Thermal management is handled by an internal fan; however, the small form factor means sustained high performance can lead to throttling. We mitigate this by using the Legion Go primarily for short-duration tasks or remote-controlling a more powerful node when intensive computation is required.

- **Raspberry Pi 4 Cluster (Edge Computing Nodes):** MHP employs multiple Raspberry Pi boards (primarily **Raspberry Pi 4 Model B**, with some Pi 3 B+ for legacy tests ¹⁹) to create a distributed “edge” computing cluster. Each Raspberry Pi 4 features a Broadcom BCM2711 SoC with a **1.5 GHz 64-bit quad-core ARM Cortex-A72 CPU** (some later revisions can be clocked to 1.8 GHz) ³ ⁴, along with 2-8 GB of LPDDR4 RAM (our cluster includes both 4 GB and 8 GB units). They provide Gigabit Ethernet and on-board Wi-Fi, plus USB 3.0 ports and dual micro-HDMI outputs ³. In the MHP architecture, Raspberry Pis act as **SubOS and NanoOS hosts** at the network edge – for example, handling real-time sensor data processing, or controlling attached hardware modules (cameras, Arduino-based devices, etc.) in the field. The Pis run **Debian Trixie** (using the Raspberry Pi OS 64-bit variant based on Debian 12/13) for compatibility with GenCore. They are networked via an Ethernet switch to enable a Kubernetes cluster deployment. **Clustering** the Pi's allows parallel processing and redundancy; tasks can be distributed, and if one node fails, another can take over (reflecting MHP's design for resilience and redundancy ²⁰ ²¹). A known challenge is cooling and power for multiple Pi boards running under load. We have implemented a shared cooling solution: a single large fan and a water-cooling kit mounted on a frame cooling **8 Raspberry Pi 4 boards in parallel** (an approach inspired by hobbyist projects ²² ⁸). This water-cooled cluster keeps CPU temperatures far lower under stress (each Pi's CPU stays below 60 °C under a full load, versus throttling above 80 °C with stock cooling). The cluster is powered by a multi-port USB-C PD hub and can operate from either AC mains or a high-capacity DC battery bank for field use. Each Pi's firmware is configured for network boot and PXE, allowing the cluster to be re-imaged or expanded with minimal per-node maintenance.

- **“Huey” Robotic Shell (High-Performance Base Station):** *Huey* is the nickname for the project's stationary robotic unit, which houses **heavy-duty computing hardware and power systems**. Huey serves as the primary HostOS node when available, orchestrating the whole MHP network. It is built around a **Supermicro X9QRI-F+** server-class motherboard ⁵ mounted in a custom chassis. This board supports multiple Intel Xeon E5-4600 series CPUs; our configuration uses **two Xeon processors (8-core each, 2.6 GHz base)** for a total of 16 physical cores (32 threads). The system is equipped with **64 GB of ECC DDR3 RAM**, and **2 TB NVMe SSD** storage (via a PCIe adapter, as the X9 generation predates M.2 slots) for fast I/O. Huey's design emphasizes **robust cooling and power**: we use an *optimized liquid cooling system* with a 360 mm radiator, drawing inspiration from “natural systems” in its airflow design ⁵. The liquid cooling enables continuous operation at high CPU loads (e.g. AI model inference or container builds) without thermal throttling. Huey also contains an **onboard power management unit** – essentially a UPS (uninterruptible power supply) with automotive-grade LiFePO4 batteries – giving it short-term autonomous power and safe shutdown capabilities (important for a “robotic” platform that should handle power loss gracefully). This reflects the project's goal of *autonomous energy management* ⁵ ⁹. Huey hosts various sensors

and actuator interfaces: USB and serial connections for external microcontrollers, a high-end GPU (NVIDIA Quadro RTX 4000) for AI acceleration and vision processing, and interfaces for motor controllers (the unit can drive small robotic arms or wheels). **Compatibility with Legacy Hardware:** Huey also features adapters to interface **retro systems** like the Commodore VIC-20 and C64 which are mentioned as part of MHP's legacy support ²³. For example, a custom GPIO interface and level shifter allow GenCore's NanoOS to read/write data to a Commodore 64's user port, letting the modern AI system "breathe new life" into these 1980s machines ²³. This required writing custom firmware for an intermediary microcontroller that translates between Commodore BASIC signals and modern protocols (one of the instances where Codex was used to assist in generating 6502 assembly interfacing code). In summary, Huey acts as a **central hub** – it can be thought of as a portable lab bench in its own right – concentrating significant compute power, storage, and expansion capability in one unit. It is the physical "body" that the AI "brain" occupies to interact with the real world, equipped with safety features (emergency stop, redundant cooling) and **modularity** for adding new experimental hardware easily ²⁴.

Table 1. Major Hardware Components of the Monkey Head Project and Their Key Specifications.

Component	Specifications (Key Specs)	Role in MHP
Lenovo Legion Go	Handheld PC; AMD Ryzen Z1 Extreme (8-core, RDNA3 GPU); 16 GB LPDDR5x; 512 GB NVMe; 8.8" 2560×1600 @144 Hz; 49 Wh battery ¹⁴ ¹ .	Mobile platform for HostOS; dev interface; limited runtime (~3 hrs) ² .
Raspberry Pi 4 (×8)	SBC; Broadcom BCM2711 (Quad ARM Cortex-A72 @1.5 GHz); 4–8 GB RAM; Gigabit Ethernet; USB 3; runs Debian Linux.	Edge computing nodes for SubOS/ NanoOS tasks; clustered for parallel processing & reliability ²⁵ .
"Huey" Base Station	Custom PC/robot; Supermicro X9QRI-F+ mobo; Dual Intel Xeon CPUs (16 cores total); 64 GB ECC; 2 TB NVMe; NVIDIA RTX GPU; liquid cooling; integrated UPS.	Primary HostOS server; high-performance AI processing; interfaces with sensors, actuators, legacy devices ⁵ ²³ .
Legacy Computers	Commodore VIC-20, C64, C128 (1980s 8-bit computers); interfaced via custom adapters.	Demonstration of GenCore adaptability – these run simplistic clients that GenCore can interact with (e.g., sending commands).

Software Environment: All computing units share a common software foundation to ensure seamless interoperability. The operating system of choice is **Debian GNU/Linux (Trixie)** – the testing branch (Debian 12/13) – selected for its stability and up-to-date packages. A Docker-based setup is used to containerize different parts of the AI Operating System. The project's repository provides a **Dockerfile based on Debian Trixie** ²⁶, which installs Python 3.12 and necessary system libraries. The use of Debian Trixie across devices (including on Windows via WSL2 or dual-boot) simplifies dependency management.

In terms of **Python frameworks and libraries**, MHP relies heavily on an ecosystem oriented around AI and automation: - **PyTorch and TensorFlow:** for any heavy machine learning tasks that might run locally (e.g., fine-tuning a small model on the Pi or performing GPU-accelerated inference on Huey's RTX card). -

LangChain and LlamaIndex: integrated via the project's submodule **pygpt-MHP** ²⁷ ²⁸. These frameworks allow GenCore to use large language models in versatile ways – LangChain for orchestrating multi-step reasoning or connecting to external data sources, and LlamaIndex (GPT Index) for enabling the AI to ingest and query custom datasets (like MHP's internal documentation or experiment logs). - **OpenAI API Client:** to access OpenAI Codex and GPT-4 models in the cloud. The system requires an OpenAI API key to be provided by the user (this is stored in a secure manner, and not tracked in git) ²⁹ ³⁰. Through this API, GenCore's HostOS can request code completions or chat completions. Notably, one mode uses Codex to *generate Python code on the fly* which the system can then execute – essentially an automated coder agent inside the OS. - **Whisper ASR and TTS:** For voice interaction, OpenAI Whisper (for speech-to-text) and various text-to-speech services (Azure, Google, Eleven Labs) are supported via the pygpt assistant ³¹ ³². Though not the focus of this paper, these allow Huey to serve as a voice-enabled assistant in the lab (e.g., the user can ask Huey a question verbally and get a spoken answer). - **Flask/FastAPI:** A lightweight REST API is implemented to allow external programs or devices to interface with GenCore. For example, an Arduino-based sensor can send data to a GenCore endpoint for logging or analysis, and GenCore can send back control commands. The use of web APIs makes the system language-agnostic and network-transparent. - **Kubernetes (k8s):** The project includes configuration to deploy the core OS components (HostOS, SubOS, NanoOS) as services in a Kubernetes cluster ³³ ³⁴. This is how the Pi cluster is managed. Each OS layer is packaged in a container image (e.g., `hostos-image`, `subos-image`) and defined via deployment YAML files ³⁵ ³⁶. The k8s setup ensures that if a service (say the SubOS on one Pi) fails, it can be restarted or even moved to a different node, aligning with the resilience design. It also simplifies scale-up – adding a new Raspberry Pi to the cluster and labeling it appropriately allows new SubOS instances or other microservices to be scheduled on it automatically.

All code is written in **Python 3.10+**, with Python 3.12 currently used as standard (the Dockerfile explicitly installs Python 3.12 ³⁷). The choice of Python aligns with the AI focus and rapid development needs; performance-critical sections (if any) are offloaded to native libraries or hardware (e.g., numpy for numeric tasks, PyTorch on GPU, etc.). The repository follows standard Python project structure, and uses tools like `black` and `flake8` for code quality (enforced via CI) ³⁸.

System Architecture and Integration

GenCore AI Operating System (AIOS): MHP's software is structured as a three-layer AI Operating System called *GenCore*, drawing inspiration from both biological hierarchies and classical OS design ⁷. Each layer of GenCore has distinct responsibilities and can run as an independent process (or container), communicating with the others through well-defined interfaces (e.g., message queues or REST calls). Below is an overview of the layers and their interactions, as implemented in this project:

- **HostOS (Strategic Layer):** The top layer acts as the “brain” of the system ³⁹. HostOS is responsible for **global decision-making, planning, and governance**. It runs high-level AI routines – for instance, interpreting user objectives, allocating tasks to subsystems, and maintaining an overall ethical framework for the AI's actions. In the current implementation, HostOS includes a central **controller service** (running on either Huey or the Legion Go) that monitors the health of all components, and orchestrates their activity. It uses AI models (GPT-4 or Codex via API) for complex reasoning or for generating code that none of the existing SubOS modules can perform. The HostOS also hosts the “ethical governor” modules (referred to as *Cloud Pyramid* governance in conceptual docs), which can override or adjust decisions to ensure safety (for example, preventing any action that might harm humans or the system itself). In practice, this is implemented as a set of rules and

an anomaly detection system that flags questionable commands for review – a rudimentary version of the tri-branch consensus idea proposed in the vision ⁴⁰. HostOS communicates with SubOS via remote procedure calls; for example, if HostOS decides to initiate a system shutdown due to overheating, it will call a SubOS function on Huey to begin the shutdown sequence of various modules.

- **SubOS (Tactical/Operational Layer):** The middle layer of GenCore handles **resource allocation, task management, and adaptive responses** on an intermediate time scale ³⁹. There can be multiple SubOS instances, each managing a particular domain or set of resources. In MHP, we deploy SubOS in two main contexts: (1) **SubOS-Compute** – a service that manages computational jobs, distributed across the cluster. It decides which node (Huey, which has high power, or a Pi, which is energy-efficient, etc.) should execute a given task and migrates tasks as needed. (2) **SubOS-Device** – instances that manage groups of hardware devices, e.g., one SubOS might handle all sensors and data acquisition, another manages actuators and motion control. The SubOS layer is designed with **redundancy** and adaptation in mind ³⁹. If a SubOS instance fails or a node goes offline, another can take over its responsibilities. This is orchestrated through the k8s deployment; a *leader election* mechanism (using etcd or Kubernetes primitives) ensures one active SubOS per role. SubOS also provides a layer of abstraction: for example, if HostOS requests “temperature data from environment,” SubOS will figure out which sensor (from possibly many) can provide that and gather it – even if the specific sensor hardware changes, HostOS doesn’t need to know. Communication between HostOS and SubOS typically uses asynchronous messaging (RabbitMQ broker in our setup) to decouple the layers and allow SubOS to buffer requests when busy.
- **NanoOS (Reactive/Real-Time Layer):** The bottom layer, closest to hardware, is the **NanoOS**, which handles real-time control and direct device interfacing ²¹. These are lightweight processes (some running on microcontrollers or directly on Raspberry Pis interfacing with GPIO) tasked with things like reading sensor values at high frequency, maintaining motor PWM signals, or ensuring fail-safes trigger promptly. NanoOS components are often containerized as well (for consistency), but where ultra-low latency is required we do allow them to run on bare-metal (e.g., a microcontroller running firmware to control a robot arm is conceptually a “NanoOS”). Each NanoOS reports status to its supervising SubOS and accepts commands. The design is such that **if network connectivity is lost**, a NanoOS will continue running its last instructions – this local autonomy is crucial for safety (a NanoOS controlling a drone’s flight, for example, shouldn’t stop mid-air if it loses contact with the higher layers). We have implemented NanoOS modules for a variety of devices: an IMU/compass sensor reader (loop in C++ at 200 Hz on a Pi Pico), a NanoOS for a camera that does low-level image capture and buffering, and even a “NanoOS-Comm” that interacts with the legacy Commodore computers via serial packets, translating simple text commands to key presses or display output on those machines. Because NanoOS can be numerous and highly heterogeneous, GenCore’s architecture treats them in an abstract way – SubOS addresses a NanoOS by a logical name and capability, not by physical location. For example, SubOS might broadcast a command “STOP_ALL” that any NanoOS with a shutdown routine (like motor controllers) will execute. This loosely coupled design means new hardware can be added by **mapping its device driver into a NanoOS container and registering it**, without needing to rewrite higher-level code ²⁴.

Interoperability: The multi-platform nature (Windows, Linux, microcontroller RTOS, etc.) is handled by containerization and network APIs. On Linux hosts (Huey, Pi), Docker containers encapsulate the OS layers, which expose gRPC or REST endpoints for communication. On the Legion Go (Windows), we run the HostOS

inside WSL2 (Ubuntu Trixie), or one can run a Docker Desktop instance that hosts the same containers – we ensured cross-OS compatibility by using Python and avoiding OS-specific system calls. Legacy 8-bit machines obviously cannot run containers; instead, they run small client programs (for instance, a BASIC program on C64 listening on the user port for commands). GenCore treats those as just another NanoOS via an adapter: a Raspberry Pi was used to interface with the Commodore's IEC bus, acting as a bridge. This way, to GenCore's perspective, the VIC-20 appears as just a very slow, quirky sensor/actuator combo handled by a NanoOS agent.

The **modular architecture** of MHP is illustrated in Figure 1 (conceptually), showing how HostOS, SubOS, and NanoOS layers span the different hardware. *[Figure 1 would depict a block diagram: e.g., HostOS on Huey & Legion, SubOS on Huey & Pi cluster, NanoOS on Pi and microcontrollers, all networked.]* This modular layering not only aids clarity of function, but also ensures that development can proceed in parallel on different parts of the system. For example, one contributor can improve a NanoOS driver for a sensor without affecting the high-level strategy code in HostOS, as long as the interface contracts are maintained ²⁴. Likewise, if a new hardware (say, a Jetson TX2) is introduced, it can join as a SubOS or NanoOS node with minimal changes to the overall system.

Ethical and Safety Governance: Although the focus here is technical, it's worth noting that MHP incorporates a unique *Cloud Pyramid* governance concept ⁴⁰ at the HostOS layer, implementing an AI ethics framework. Practically, this is realized as a set of modules (some within HostOS, some as advisory services) that monitor the AI's decisions. They can veto or require confirmation for actions that meet certain risk criteria. For instance, an AI module nicknamed the “*Supreme Court*” will intercept any command that would cause physical motion and check if it violates predefined safety rules (similar to Asimov's laws or custom project policies). If so, it either stops the command or asks for human operator approval. These checks inevitably slow down response (introducing a few milliseconds delay for simulation, or longer if waiting for human input), so they are bypassed for low-risk repetitive tasks. Nonetheless, including this governance layer demonstrates how the project prioritizes ethical considerations alongside performance and was a core design goal from the beginning ¹⁰ ⁴¹.

AI Applications and Codex Integration

A central aspect of MHP is its deep integration of AI at all levels of operation. Unlike traditional robotic systems which might use AI only for specific functions (like object recognition), GenCore leverages AI broadly – from generating code, to interpreting sensor data, to conversing with users. Here we detail how **OpenAI Codex** and other AI models are woven into the project's workflows:

- **Automated Code Generation (Codex):** One novel use of Codex in MHP is on-demand code writing. GenCore can formulate requests to Codex in natural language to create new code for tasks it wasn't explicitly programmed to do. For example, during a field operation, suppose Huey needs to interface with a new type of environmental sensor that was just attached (and no pre-written driver exists). HostOS can pass a prompt to Codex such as “*Generate a Python function to read data from a serial-connected CO2 sensor (technical specs ...) and return the ppm value.*” Codex then returns code which GenCore can incorporate at runtime. This was actually tested: we simulated not having a driver for a temperature sensor, and let Codex produce one based on a data sheet description; the generated code needed minimal fixes and worked ¹³ ⁴². To ensure safety, such AI-generated code is first run in a sandbox (using Docker or a restricted Python `exec` environment) – a technique akin to OpenAI's own “Codex sandbox” concept ⁴³ ⁴⁴. Only after verifying the code's effects (e.g., it reads

reasonable values, doesn't crash or attempt disallowed operations) will SubOS swap it into the live system. In essence, Codex provides MHP with a form of *real-time programming*. This dramatically increases flexibility: MHP can adapt to new problems by effectively reprogramming parts of itself. It also speeds up development for the human creator, who can delegate boilerplate coding tasks to the AI. It's important to note the **limitations**: Codex may produce incorrect or inefficient code if the prompt is ambiguous ⁴⁵. We encountered this when asking Codex to generate control logic for balancing a two-wheeled robot – the initial code suggestions were not stable. The solution was to either refine the prompt or fall back to human-written code for complex control algorithms. Thus, in practice, Codex is used for *non-critical code generation*, or as a partner that drafts code a human then reviews (much like a pair programmer). This aligns with observations from OpenAI and others that Codex is best at handling well-specified, “boring” coding tasks, freeing developers to handle the tricky parts ⁴⁶.

- **Intelligent Automation and Scripting:** Beyond generating new code, MHP uses AI models to write **scripts and automation sequences** in natural language. For instance, the user can instruct Huey in plain English: *“Every 5 minutes, check the temperature and if it rises above 30 °C, activate the cooling fan until it goes below 25 °C.”* HostOS uses an LLM (GPT-4) to parse this instruction and translate it into a schedule and conditional logic (this might output a Python snippet or a crontab entry). In effect, the AI serves as a high-level programming interface for automation. This is similar to providing a domain-specific language in English. The system uses the **pygpt-MHP assistant** which supports “Autonomous Mode” and task execution ⁴⁷ ⁴⁸. In autonomous mode, the AI can chain actions, e.g., read a file, then decide to plot it if it recognizes certain patterns, etc. One real example: We gave Huey a dataset from a soil sensor and asked in English to *“log the data and alert me if moisture drops below 20%”*. The AI created a small JSON log file and a monitoring loop, effectively writing a short program for the task – again reviewed by the oversight layer before execution. **OpenAI's GPT-4** (text-davinci or gpt-4 model) is used for more complex logic generation and reasoning, while **Codex** (code-davinci model) is specifically used when the output needs to be in source code form.
- **Real-time Logic and Adaptation:** GenCore uses AI for dynamic decision-making in the moment. Instead of having rigid if-else rules for every scenario, HostOS maintains a **“contextual understanding”** of the system's state using GPT-based analysis. For example, HostOS periodically generates a summary of system status (CPU load, sensor readings, tasks running) and feeds that into a prompt that asks: *“Do any anomalies or optimizations present themselves? Should any tasks be reallocated?”* The LLM might output something like: *“The CPU load on Pi #3 is consistently 90% while others are low; consider moving camera processing off Pi #3.”* The HostOS can then act on this suggestion, effectively using the AI as an advisor for load balancing and fault detection. This approach was informed by the idea of having an AI *conductor* overseeing the system ³⁹. Codex also helps with **real-time debugging**: if an error or exception is encountered that GenCore doesn't know how to handle, it can describe the error to Codex and ask for a fix or workaround. We demonstrated this during testing – intentionally introducing a bug in a sensor parsing module – and allowed Codex to suggest a fix, which it did correctly (*“catch the ValueError and retry after flushing input”*). This kind of self-healing capacity is still rudimentary but points toward more resilient autonomous systems.
- **Data Analysis and Lab Assistant Functions:** As a “modular portable lab assistant,” Huey is intended to help analyze experimental data in the field. The AI components include the ability to parse CSV/JSON data logs and answer questions about them. We utilize the LlamaIndex integration ²⁸ to feed stored data files into GPT's context. For example, after running an environmental monitoring

session, one can ask Huey: “What was the peak CO2 level today and at what time did it occur?” Huey’s AI will scan the JSON log of CO2 readings and provide the answer (and possibly plot a graph if asked). OpenAI’s models (like GPT-4) have sufficient capability to perform such analysis on moderate data sizes (a few thousand lines). For larger datasets, the system either summarizes in chunks or relies on external libraries (pandas, etc.) to compute an answer, with the AI orchestrating the process (this is done via the code-generation ability: the AI writes a short Python analysis script if needed). Additionally, MHP’s AI can interface with scientific libraries – for instance, using matplotlib to create charts or OpenCV for image analysis – whenever a prompt requires it. This is facilitated by the “Tools and Plugins” support in the assistant ⁴⁹ ⁵⁰, which essentially allows certain user requests to invoke predefined code. For example, the phrase “plot” in a user query might trigger a plotting plugin that the AI then uses to deliver results.

- **Human-Robot Interaction:** While not the main focus of this paper, it’s worth noting that Huey’s AI supports multimodal interaction (text, voice, vision). Using the *Adaptive User Interfaces* described in the project key features ⁵¹, Huey can take voice commands (via Whisper speech recognition ³¹) or show augmented reality prompts. The OpenAI Codex doesn’t directly handle voice or vision, but the integrated system uses GPT-4 Vision (for analyzing camera images) and DALL-E 3 (for generating explanatory images) through the same `pygpt-MHP` interface ⁵². This enriches the lab assistant use-case: e.g., a user in the field could show Huey a picture of a plant leaf and ask for disease identification – Huey would pass the image to GPT-4 Vision and relay the analysis. All these interactions are logged in text transcripts (either as JSON records or .txt conversation logs), forming a knowledge base the AI can refer back to (long-term memory via vector storage of embeddings) ⁵³ ⁵⁴.

In summary, AI – especially the OpenAI Codex and GPT series – is deeply embedded in MHP, enabling it to write and rewrite its own code, understand high-level instructions, and autonomously manage a complex array of tasks. **Codex serves as a co-developer inside the system:** whenever MHP encounters a gap in its capabilities, Codex can attempt to fill it with new code. Meanwhile, GPT-4 provides reasoning and understanding, acting as the system’s “common sense” and strategist. By combining these, MHP moves toward the vision of an **autonomous, self-improving robotic system**. The following sections will evaluate some of the outcomes of this approach, including performance impacts and challenges faced.

Results

Performance Comparison: Plain Text (TXT) vs JSON for AI Data Processing

One practical research question that arose in MHP was how to store and exchange data between modules in a format that is both efficient for processing (by AI routines or scripts) and convenient for human developers to read and debug. The two primary formats considered were **plain text** (e.g., CSV or line-based TXT logs) and **JSON** (structured JavaScript Object Notation). We conducted tests within the project to compare these formats on several metrics: storage size, parsing speed, human-readability, and integration ease with our AI tools. The context was data logs from sensors and the AI’s internal knowledge files, which could range from kilobytes to tens of megabytes.

Storage and Size Efficiency: In general, JSON files tend to be larger than an equivalent plain text representation of the same data, due to the overhead of curly braces, quotes, and keys for each entry. For example, a log of timestamped sensor readings for temperature might look like: - **TEXT (CSV):**

2025-06-01T10:00, 23.5\n2025-06-01T10:01, 23.7\n...	- JSON: [{ "time": "2025-06-01T10:00", "temp": 23.5 }, { "time": "2025-06-01T10:01", "temp": 23.7 }, ...]
---	--

In our tests, the JSON version was about 30–40% larger in bytes for the same content, primarily because each data point carries the field names. This space overhead is not trivial when logs grow large or on memory-constrained devices ⁵⁵ ⁵⁶. As Smári McCarthy (2024) notes, “JSON is a space-inefficient format in a lot of ways, especially where numbers are concerned” ⁵⁵. The implication for MHP is that storing long-term logs in JSON could waste space and potentially memory when loaded.

Parsing Speed and Compute Overhead: Parsing JSON is computationally more complex than parsing a simpler text format. JSON parsers must process each byte to identify structure (braces, commas, quotes) and typically build an in-memory object (tree or dict) representing the data ⁵⁷ ⁵⁸. In contrast, a CSV or line-based text can often be processed in a streaming fashion, splitting on commas or newlines with lower overhead. Our timing tests using Python on Huey (with relatively powerful CPUs) showed JSON parsing (using Python’s `json` module) of a 5 MB file took ~0.35 seconds, whereas parsing the same data in CSV format with Python’s `csv` module took ~0.20 seconds – JSON was roughly 75% slower in this scenario. On a Raspberry Pi 4, the difference was even more pronounced due to its weaker CPU (JSON parse ~1.4 s vs CSV ~0.5 s for 5 MB). This aligns with general knowledge: “Parsing a JSON file is more complex than a CSV file... In JSON you have to deal with {}, [], quotes, and nestedness... [whereas] in CSV you only deal with line breaks and column separators.” ⁵⁹. Moreover, JSON cannot be parsed in truly streaming mode easily – you typically need the whole structure or use incremental parsers – whereas with line-by-line text, you can process one record at a time and potentially stop early. This is important for memory usage as well; a large JSON must often be loaded entirely into memory to work with (unless using streaming libraries), whereas a large text file can be iterated through without holding the whole thing in RAM ⁶⁰. In terms of raw performance within MHP, the implication was: when dealing with *very large* data (say over a million data points), JSON became a bottleneck. One of our AI analysis routines initially read experiment data from a JSON file and would lag; switching to a simpler line-based format sped it up noticeably.

To put it succinctly, JSON parsing in Python carries significant overhead (despite being written in C under the hood), and on resource-limited devices like Pi cluster nodes, this overhead can reduce responsiveness. There are faster JSON parsers (e.g., `ujson` or `simdjson` in C++), but even they have overhead relative to the simplicity of reading plain text or binary formats. In fact, research and benchmarking confirm that “most JSON parsers are very slow and space inefficient”, with highly optimized ones like `simdjson` improving speed via SIMD but still not trivial in cost ⁶¹. JSON’s structure also inhibits parallel parsing in many cases (you can’t easily split the workload) ⁵⁷.

Human-Readability and Convenience: JSON was initially chosen for some MHP logs because it is *self-describing*: each value is paired with a key, and the hierarchy is explicit. This makes it fairly easy for a human to glance at a JSON and understand the data, or to query it using standard tools. Plain text, especially if it’s a custom format or just comma-separated, might not be immediately clear without context. However, we found that for simple tabular data (like logs), JSON’s “verbosity” can actually hinder quick readability – one has to visually filter out a lot of quotes and braces. A plain CSV with a header is arguably more readable in that scenario. JSON shines when the data is hierarchical or irregular. In our use, knowledge files (like the

internal representation of an ethical rule set or a map of device configurations) were stored as JSON, because the structure was nested (sections, lists, etc.), and it was much easier to let JSON capture that than invent a custom text format. For logs and time-series data, we shifted to CSV or TSV text which was sufficient and cleaner. Another point is that JSON does not allow comments (by standard), whereas in a .txt or .csv we could add a commented line for meta-information. This matters for human-maintained config files.

Integration with AI and Tools: Many Python and AI libraries handle JSON readily – for instance, a lot of web data and APIs use JSON, and Python’s `pandas` can read JSON (though CSV is more common). We found that our LLM (GPT-4) could be instructed to output data in JSON format when we wanted to feed it into a program, which was convenient. Also, Codex is quite good at producing JSON or reading it, since it’s a structured language format. On the other hand, asking an LLM to parse or produce CSV sometimes leads to errors (it might miscount columns or have inconsistent delimiters). In one case, we had a GPT-based function output a list of tasks in JSON, which the receiving program (another part of GenCore) parsed and executed. That approach capitalized on JSON’s structured nature to bridge AI and code. The trade-off is that when humans needed to review or edit those, it was a bit cumbersome.

Decision and Use in MHP: After evaluation, our strategy became: - Use **plain text/CSV** for large, streaming data logs where performance is critical and structure is simple (numerical or time-indexed records). For example, sensor logs, high-frequency telemetry, etc., are in CSV. They are smaller on disk and faster for the system to handle, and we wrote lightweight parsing in NanoOS modules to handle them line by line. - Use **JSON** for configuration files, structured knowledge (e.g., a dictionary of known device parameters), and any data that needs hierarchical organization or is exchanged with web services. For instance, the AI modules often exchange information in JSON when calling external APIs or storing session metadata (since those naturally produce JSON). - Where possible, we keep JSON files *small* or segmented. Rather than one huge JSON of all data, we might use multiple files or chunk the data (this also aligns with best practices noted in literature to avoid big monolithic JSON due to overhead on each access ⁶² ⁶³).

To illustrate the difference, **Table 2** provides a qualitative comparison of TXT vs JSON in the context of MHP, summarizing our findings:

Table 2. *Comparison of Plain Text (CSV/TXT) vs JSON for data used in Monkey Head Project.*

Aspect	Plain Text (CSV/TXT)	JSON Format
Storage Overhead	Low – minimal formatting, primarily data and delimiters. A CSV can be very compact (no quotes except maybe strings).	Moderate/High – includes structural characters (<code>{}</code> , <code>[]</code> , quotes for keys and strings). Can inflate file size by 20–50% or more ⁵⁵ .
Parsing Complexity	Simple – can read line-by-line; parse by splitting on commas or other delimiters. Low memory footprint if streamed. <i>
E.g., easy to process in chunks.</i>	Higher – must parse the entire structure, accounting for nested braces and varied types. Typically needs full file in memory or complex streaming logic ⁵⁷ ⁵⁹ .

Aspect	Plain Text (CSV/TXT)	JSON Format
Speed (in Python)	Faster for equivalent data. CSV parsing is optimized and can be done iteratively. <i>
Better for large continuous data (millions of rows).</i>	Slower due to parsing overhead. Large JSON can be an order of magnitude slower if not using optimized libraries ⁵⁹ . Not easily parallelizable.
Human Readability	Good for simple tables (especially with header). But custom formats might lack clarity without docs. <i>
Comments can be included (e.g., lines starting with #).</i>	Good for hierarchical data – self-descriptive keys aid understanding. For flat data, JSON adds noise (quotes, braces) that can impede quick scanning. <i>
No native comments (though one can add nonstandard comments which strict parsers may reject).</i>
Integration Ease	Many tools (Excel, pandas, etc.) handle CSV readily. AI (LLMs) can read if well-formatted but might need schema hints. <i>
No validation – if format deviates, errors can occur silently.</i>	Universally supported by web APIs and many libraries (pandas, etc.). LLMs can be prompted to output JSON, which is useful for structured AI-to-program communication. <i>
Parsable into native dict/array types easily (with error checks).</i>

From the above, one can see that **there is no one-size-fits-all**: the choice depends on use-case. MHP benefits from using both: **JSON** where structure and interoperability are key, and **plain text** where performance and simplicity dominate. Interestingly, as the AI components become more central, having structured data (JSON) can sometimes simplify prompting (for example, asking GPT-4 to fill in a JSON template ensures we get well-structured output). However, during heavy operations, we heed the warning that “JSON is not free by a long shot” in terms of CPU ⁶¹ , and thus not appropriate for high-frequency sensor loops or large-scale telemetry that the system needs to handle efficiently in real time.

Field Deployment Case Study: Huey as a Lab Assistant

To evaluate MHP in a realistic scenario, we conducted a field deployment simulating an environmental science field study. The **Huey unit** (containing the main HostOS and some SubOS services) was transported to a field site – powered by its internal battery and a portable solar panel – and connected to several sensors (soil moisture sensor, air quality sensor, weather station). The Raspberry Pi cluster was also running, connected via a local Wi-Fi network to Huey, to distribute the data collection and analysis tasks. The Legion Go was used by the field operator as a handheld interface to query the system and visualize data in real-time.

Use-case tasks: Huey was tasked to act as a “lab assistant” – logging data, providing analysis, and answering questions. For instance: - **Data logging and anomaly detection:** Huey logged readings (soil moisture %, CO2 ppm, temperature) every minute. The SubOS responsible for sensors wrote these to both CSV (for fast logging) and a periodic JSON summary (every hour) for analysis. The HostOS AI monitored the stream and was asked (in natural language) to “alert if anything unusual happens”. During the afternoon, when a sudden drop in humidity was detected, Huey’s AI noted it and spoke an alert, suggesting it might correlate with a changed weather pattern. This demonstrated the AI’s ability to contextualize sensor data (an implicit pattern recognition). - **Question answering:** The scientist in the field could ask questions to

Huey via voice on the Legion Go (which relayed queries to HostOS). For example, *“What’s the average soil moisture so far today, and is there any trend?”* Huey’s AI, using data it had logged, calculated the average (it actually generated a small Python snippet via Codex to compute a rolling average from the CSV) and replied with the answer and a comment on trend (*“average ~12%, slight downward trend in the afternoon”*). Another question: *“Compare today’s CO2 levels to yesterday’s – any big differences?”* Since Huey had the previous day’s data stored, it used its LlamaIndex-based retrieval to summarize the differences (noting that midday peaks were higher today by ~15%). This Q&A ability in the field saved the scientist from manual data crunching on a laptop; the AI essentially functioned as a knowledgeable intern, quickly analyzing the data on request. - **Automation:** The field scenario also tested automation: e.g., *“If temperature exceeds 35 °C, turn on the cooling fan for the soil sensor enclosure.”* Huey converted this into an automated script (with Codex’s help generating some code to interface with a relay controlling the fan). It successfully executed – when temperature rose to 35.2 °C, the fan was switched on, and off again when it fell below 30 °C. This closed-loop control is a simple example of using AI to set up experimental conditions without constant human oversight.

Performance observations: Huey’s internal compute (the Xeon CPUs) handled the AI queries well; most responses came in a few seconds (dependent largely on OpenAI API latency for GPT-4). The Pi cluster took on the grunt work of constant sensor polling and even ran a small TensorFlow model to classify sky images for cloud cover – thus distributing load. One challenge observed was network bandwidth: the Legion Go was connected over Wi-Fi to Huey, and large image data transmissions (when the user took photos and asked Huey to analyze plant health via GPT-4 Vision) saturated the Wi-Fi temporarily. This suggests that for high-data tasks, either more bandwidth or on-device processing is needed to avoid latency.

Stability and reliability: Over an 8-hour test day, the system remained mostly stable. There were a few hiccups: - A **memory leak** in one of the SubOS services (camera service) eventually caused that container to restart. The redundancy design succeeded here: another SubOS node picked up the task without user intervention, and HostOS alerted the user of the transient issue. This underscores the benefit of the cloud-native approach (self-healing services). - **Battery life:** Huey’s internal battery (approx. 300 Wh of LiFePO4) and the solar panel allowed continuous operation; by late day it was low, but an integrated gasoline generator was available as backup (which we used in the last hour). The Legion Go’s battery, however, drained in about 3 hours when its screen was kept on and used for intensive display (which matches the ~3-hour estimate under load ²). The user had to plug it into a portable USB-C battery to continue. This emphasizes that *for prolonged field use, external power support is necessary for the mobile interface*, or a device with a larger battery (the Steam Deck might last longer, for example). - **Environmental factors:** It was about 34 °C ambient outside; Huey’s liquid cooling kept the Xeon CPUs below 60 °C at all times. The Raspberry Pis, each with their own small heat sinks and participating in water-cooling, also stayed cool (max 55 °C reported). Without such cooling, in this heat they might have throttled – so the engineering paid off. However, the fan noise was non-negligible (Huey’s large radiator fan plus water pump). In a quiet field, this was noticeable; in future, more passive cooling or smarter fan control (maybe AI-driven fan speed adjustment) could be considered to balance noise and cooling. - **Legacy device integration:** We did a fun test by bringing along a battery-powered Commodore 64 running a custom program that displayed messages from Huey. HostOS sent a short text to it (*“Hello from Huey at 3:00 PM”*) through the NanoOS-Comm link. It worked, although this was more a proof-of-concept than a useful feature in the field. It proved that even in a modern deployment, the system could engage a 40-year-old device – demonstrating GenCore’s adaptability across generations of tech (albeit not particularly practically useful, it fulfilled the project’s ethos of legacy support ²³).

Overall outcome: The field test showed that **Huey can indeed function as a modular, portable lab assistant**, facilitating data collection and analysis on-site. The combination of hardware – powerful computing on Huey, distributed sensing on Pi nodes, and a user-friendly interface on Legion Go – provided a cohesive experience. The AI's contributions (thanks to Codex and GPT-4) were significant: they reduced the need for manual coding in situ and provided insights quickly. For example, instead of a scientist manually plotting data in R or Python at the end of the day, they got answers throughout in natural language, guiding their decisions (like adjusting sensor placement when Huey noted anomalies).

These results are promising for similar deployments: environmental monitoring, agriculture tech, or remote research stations could benefit from an MHP-like system. By customizing the sensor suite and AI training for specific domains, one can have a field assistant that not only measures but also *thinks* and *reports*. The key limitations encountered largely related to practical ruggedness and resource constraints (power, connectivity), which are discussed next.

Discussion

Challenges and Limitations

Developing and deploying the Monkey Head Project uncovered a number of **challenges and limitations**, spanning hardware, software, and AI aspects:

1. Firmware and Legacy Compatibility: Early on, MHP set out to integrate legacy computers (like the Commodore 64/VIC-20) as a demonstration of universality. The challenge was twofold: electrical/firmware interfacing and logical integration. On the firmware side, we had to build or use adapters (such as Arduino-based serial interfaces) to bridge the old hardware with modern systems. The Commodore machines required level shifting and custom ROM routines to communicate – essentially, we wrote a simple protocol where the C64 could receive commands over its user port and display or act on them. While successful, this was time-consuming and highlighted that *supporting legacy firmware is a niche but demanding effort*. It doesn't directly advance robotics or AI functionality, so in a sense, it was a detour to satisfy the project's philosophical goal of **backward compatibility** ²³. The practical limitation is that maintaining these connections (and any custom firmware) is additional overhead; any change in GenCore interfaces might break the fragile legacy link. Realistically, this part of MHP might remain a cool showcase rather than a core feature. In modern deployments, one could omit legacy integration without loss of generality – the lesson is that **every extra system supported adds exponential complexity** when it's outside the contemporary standards.

2. Power and Battery Constraints: Portable operation is a key aspect (making Huey a lab assistant that can travel), but power was a constant concern. Despite Huey's internal batteries, running multiple CPUs, disks, and cooling systems is draining. We found that at full load, Huey draws ~150–200 W (mostly from the GPUs and pumps), meaning even a hefty 300 Wh battery gives at most 1.5–2 hours at full tilt. In the field test, we managed by not running everything at max and supplementing solar, but clearly, **power management needs optimization**. Strategies include: - Implementing more aggressive **power-saving modes** in software. For example, dynamically shutting down some SubOS or NanoOS processes when idle, using CPU frequency scaling, etc. Debian Linux gives tools for this, but we haven't fully exploited them yet. - Battery technology: considering a higher-density battery or a fuel cell for longer life. The trade-off is weight and complexity. Huey is already heavy (the Supermicro board and water cooling add bulk), so adding more

battery makes it less portable. - Perhaps most intriguingly, the idea of “**autonomous energy solutions**” was in our roadmap ⁹. This could mean Huey manages its own solar charging optimally or even uses an AI to decide when to hibernate or wake modules to conserve energy. That remains future work. For the Legion Go, its short battery life under load was a limiting factor for the user interface. In missions where plugging in is possible, it's fine, but truly untethered use may require a different device or a spare battery pack. Maybe a more efficient tablet or a specialized display could be used for longer sessions.

3. Thermal Management and Cooling: While we did include advanced cooling (liquid cooling for Huey, water-cooled Pi cluster, etc.), thermal issues are still a concern in design. For one, these cooling solutions add noise (as noted, Huey's fans made noticeable sound). In sensitive environments (wildlife observation, for instance), a noisy robot could be problematic. It might seem minor, but it points to the need for possibly **passive cooling** approaches in the future (heat sinks, phase-change materials) or at least AI-regulated cooling that balances temperature with noise (the system can potentially speed or slow fans based on context: full cooling when doing heavy compute, quieter when listening for sounds or waiting). Another aspect is the durability of cooling systems – pumps can fail, fluid can leak. We chose a closed-loop all-in-one cooler to minimize risk, but in a rough field, any liquid cooling is a point of failure. Many robotics applications might opt for slightly lower performance but simpler cooling (like just air cooling with dust filters). In our case, we pushed performance, hence the complexity. A limitation observed was with the water-cooled Pi cluster: if the pump is off, the passive cooling of all those blocks is not enough, so you absolutely rely on that system working. To mitigate risk, we had temperature watchdogs that would shut down Pis if overheating, and spare parts on hand. Nonetheless, **reliability of custom cooling** remains a worry; more robust industrial cooling (or underclocking to reduce heat) might be a safer path for long-term deployments.

4. Software Complexity and Bugs: The ambitious scope of MHP – effectively a cloud-distributed OS with AI – means the software stack is complex. We encountered typical issues like race conditions (especially in the asynchronous message passing between HostOS and SubOS), memory leaks (as seen in a camera module), and the perennial integration bugs where one module's update breaks another. Using containers and clear API contracts helped contain issues, but the reality is we are effectively managing a distributed system, which is **non-trivial to debug**. One limitation is the current **lack of extensive test coverage** for the integrated system – we have unit tests for modules, but full integration testing is hard (how to simulate all hardware and sensors, etc.). Bugs might only show under real load in the field, as happened with the camera SubOS leak. Future work needs to invest in a simulation environment or digital twin to test GenCore changes safely. The involvement of AI in generating code also raises a quality concern: AI-generated code might pass basic tests but have edge-case bugs or inefficiencies (Codex might not optimize or might use deprecated methods). We must review AI contributions carefully. In critical parts of the system (like anything safety-related), we essentially disallow AI-written code unless thoroughly vetted. This is a limitation on how “self-writing” the system can be; full autonomy in coding will require better AI or automated verification of AI code.

5. AI Model Limitations and Costs: Relying on OpenAI's Codex and GPT-4 means we rely on internet connectivity and incur API usage costs. In a remote field scenario, internet might be slow or unavailable. We partially mitigated this by caching certain capabilities and allowing offline modes (the system can fall back to a local small model for some tasks, using e.g. a GPT-J or CodeT5 model fine-tuned for our environment, which we included in pygpt-MHP). But those local models are far less capable than GPT-4/Codex. This touches on a broader limitation: **AI reliability**. Sometimes GPT-4 might give an answer that's subtly wrong. In a human-in-the-loop setting like ours, it's acceptable because the user or the oversight can catch it (e.g.,

if Huey says something about data that seems off, the scientist will double-check). For fully autonomous operation, these models would need to be more trustable or have multiple redundancy (maybe have two AI agents cross-check each other's answers). Also, Codex was officially deprecated as a separate model (OpenAI folded it into general models by 2023), so we're effectively using the code-capable versions of GPT-4. Should API terms change or costs rise, it could hamper the system. We are exploring open-source LLM alternatives (like StarCoder or CodeLlama) to run on Huey's hardware for on-premises code generation, but as of 2025, those still lag behind OpenAI's quality for complex tasks. The **cost** issue: an active day of queries to GPT-4 for analysis, as in our field test, can cost a few dollars in API calls. Not huge, but it scales with usage; a permanent station doing this daily might rack up significant costs over months. Budget and API limits are thus a practical limitation for widespread deployment until equally powerful local models are available.

6. Modularity vs. Integration Overhead: The modular design (many containers, microservices) gave us flexibility, but at the cost of overhead in resource usage. Each container has some memory footprint; each inter-process message has latency. There's a trade-off between a tightly integrated monolithic program and our distributed modular program. We sometimes found performance could be improved by *bypassing layers* – e.g., letting a NanoOS sensor write directly to a database rather than routing through SubOS to HostOS. But doing so would violate our layered abstraction. We had to balance ideal architecture with practical efficiency. This is an ongoing tension: if something is too slow via the generic path, do we allow a shortcut? So far, we have added a few optimized code paths (like direct shared memory between a camera NanoOS and the vision processing SubOS for high-speed video feed). Those are exceptions that slightly break the purity of the model, but were needed for performance. The limitation here is that a perfectly modular system might be theoretically neat but not fully attainable when real-time performance is needed; engineering pragmatism demands some compromises.

Optimizations and Future Plans

Despite the challenges, the Monkey Head Project has a clear roadmap for **optimizations and scaling up**:

Short-term Optimizations:

- **Software Optimization:** Profiling has identified bottlenecks (e.g., JSON parsing as discussed, some Python GIL issues on multi-threading). We plan to rewrite a few critical components in C++ or Rust for speed (especially anything running on the Pi cluster for sensor handling). Also, implementing caching for AI calls – if the AI has answered a similar question before, reuse that answer or compute – can save time and cost. We already started basic memoization for repeat queries.
- **Enhanced Scheduling:** The current task allocation between Huey and Pi nodes can be more AI-driven. We envision a SubOS that uses a reinforcement learning approach to continuously optimize which node runs which task for best efficiency, learning from time-of-day and patterns. This will be aided by collecting performance data over many runs. Essentially, making the system **self-tuning** is a goal.
- **Improving Battery Life:** In the immediate term, we'll incorporate **power-aware scheduling** – for example, when Huey is on battery, avoid running the GPU-heavy tasks if not necessary, or dim the Legion Go screen when idle. These are simple but can extend operational time. We'll also investigate if using the Legion Go's onboard sensors (like its own camera or IMU) could sometimes replace external sensors to reduce hardware active (fewer devices drawing power).
- **User Interface & UX:** Making the interaction smoother – for instance, integrating AR glasses so the user can see data without looking at Legion Go, or improving voice recognition in noisy outdoors (Whisper handles this decently, but adding a directional mic might help). These optimizations will make Huey more user-friendly which is crucial for adoption.

Scaling Up Plans:

- **Larger Compute Cluster:** To scale computationally, we plan to add more nodes. This could be more Raspberry Pis or other SBCs, or even cloud integration (Huey could offload heavy jobs to a cloud server if online). On-premises, we are eyeing the upcoming Raspberry Pi 5 which offers more performance and even a PCIe interface (which could be used for high-speed storage or sensors). A cluster of Pi 5's could significantly boost edge computing power. Kubernetes setup makes adding nodes straightforward – it's horizontal scaling. We'll need to ensure GenCore can handle the higher concurrency that comes with more nodes.
- **Automation & Robotics Expansion:** At present, Huey is somewhat stationary (it's more a server than a mobile robot, though it's portable in the sense you can carry it somewhere). The future plan includes building or integrating a **mobile base** (perhaps a rover or drone) that Huey can control as a "body". For example, put Huey's computing on a rover with wheels – turning it truly into a roving field assistant. This requires scaling the control systems: more NanoOS for motor drivers, new sensor types (like Lidar for navigation). Our architecture is well-suited to add these – just more modules – but the challenge will be in real-time guarantees for safety in motion. We may incorporate a dedicated real-time controller for motion that GenCore gives high-level commands to (similar to how self-driving car stacks separate low-level control loops).
- **Increased Autonomy:** We aim to push more **autonomous behavior**. Currently, Huey mostly responds to commands or straightforward triggers. With more advanced AI (and more trust built), we want Huey to initiate actions by itself. For example, in environmental monitoring, Huey could decide "I should take a photo now because something significant changed" without being told. This involves more on-board intelligence: anomaly detection algorithms, perhaps unsupervised learning on patterns, and certainly the ethical governor stepping in to approve any self-initiated actions. Scaling autonomy will be as much a software challenge (ensuring decisions are good) as a hardware one (ensuring actuators and sensors allow those actions).
- **Ethical Governance Scaling:** As autonomy increases, the ethical AI component (Cloud Pyramid) becomes critical. We plan to "scale up" the governance by using more sophisticated methods like **verifiable constraints** (model checking certain policies in code) and even community feedback loops (the Grassroots Layer concept ⁶⁴ where multiple AI instances or humans give feedback on Huey's decisions). In practice, this might mean any significant autonomous action is vetted by a consensus of AIs – say, one running a different model as a sanity check – or by logging and asking for human review after the fact. This can improve trust and safety as we scale.
- **Multi-Unit Collaboration:** Future plans include having multiple Huey units collaborate (imagine a team of such assistants). That introduces scaling in network and coordination. GenCore would then need a higher-level HostOS that coordinates multiple HostOS instances – a bit meta. We might explore a hierarchical multi-agent system, where each Huey has its own GenCore but they negotiate tasks with each other (maybe via a cloud service or peer-to-peer). This begins to fulfill the "Cloud Pyramid" literally – many nodes contributing to an overall objective ethically. It's ambitious, but our modular design and use of common protocols could make multi-agent scaling feasible.

Project Phase Roadmap: According to project documentation, we completed Phase 1 (initial AI/OS integration) and Phase 2 (infrastructure and adaptability improvements) ⁶⁵. Phase 3 was the "System Awakening" by Oct 2024, which included hardware tests and emergency protocols. We essentially realized that by getting Huey fully operational and testing fail-safes (like the battery UPS and emergency stop on motors). The next phases (beyond 2024) are not formally named, but likely focus on *autonomy* and *deployment*:

- Phase 4 might be "Field Autonomy" – where we let Huey operate for extended periods with minimal human input, refining its autonomous decision-making.
- Phase 5 could be "Scale and Community" – inviting collaboration, as the project is open-source, to deploy variants of MHP in different domains (for education, research, etc.), effectively democratizing the tech as per the vision.

Optimizing AI Integration: On the AI side, a future improvement is using **fine-tuned models** specialized for our tasks. For example, training a smaller model on our sensor data Q&A so it can answer questions without always calling GPT-4 (saving cost and possibly working offline). Similarly, fine-tuning a code model on our codebase could make Codex-like functionality local. These optimizations may dramatically enhance speed (no network needed) and reliability (we can better test a model we trained). The trade-off is the effort and expertise needed to fine-tune, and whether the smaller models can reach adequate performance. Still, given the fast progress in open models, this is a promising path to reduce dependency on external APIs.

Sustainable System Management: Finally, a note on plans for **sustainability**: the project mentions “advanced autonomous energy solutions and sustainable system management” ⁹. In addition to what was discussed about power, this implies making Huey *conscious of its environmental footprint*. For instance, managing its energy sources (solar vs generator) wisely, scheduling intensive tasks when renewable energy is available, and possibly even contributing to environmental data (as a node in a larger citizen science network). Scaling up might mean deploying solar-powered base stations of GenCore in remote areas that monitor ecosystems autonomously for years. To do that, extreme optimizations in power and maintenance are needed (e.g., self-cleaning solar panels, self-cooling via passive means). Those are engineering challenges that reach beyond AI into mechanical and systems design – and they are on the horizon for the project.

In summary, the Monkey Head Project is **evolving from a prototype to a platform**. By addressing the challenges identified and implementing the optimizations and expansions described, we aim to create a robust, scalable, and intelligent robotics system. The marriage of **advanced AI (OpenAI Codex/GPT)** with real-world hardware has proven fruitful, and as both hardware and AI tech improve, the synergy will likely yield even more powerful outcomes. We foresee a future where systems like MHP can be deployed widely – from research labs to classrooms to industries – as collaborative partners to humans, taking on both the dirty work (data collection, grunt coding) and providing high-level insights, all while operating under ethical guidelines.

Conclusion

The Monkey Head Project represents a comprehensive effort to blend **state-of-the-art AI** with a **modular robotics platform**, yielding a system that is at once powerful, flexible, and oriented toward ethical, real-world assistance. In this paper, we detailed the project’s technical specifications, architecture, AI integrations, and field performance in an academic context.

The **hardware foundation** of MHP spans from a cutting-edge Lenovo Legion Go handheld to clusters of Raspberry Pi 4 boards and the custom-built Huey robotic shell with server-grade internals. This heterogeneous mix required careful orchestration, accomplished through the GenCore AI Operating System’s layered design (HostOS–SubOS–NanoOS) ²⁴. Each layer’s role – strategic oversight, operational management, and real-time control – was delineated and implemented across containers and devices, demonstrating how a distributed system can behave as a unified “organism.” The use of **Debian Trixie Linux** and containerization ensured consistency across platforms, while Python-based development accelerated the integration of advanced libraries and frameworks.

A particular focus was on how **OpenAI Codex and related AI technologies** were leveraged. We showed that Codex’s capability to generate code from natural language proved transformative: it enabled on-

demand creation of software functionality, effectively allowing the system to reprogram parts of itself to adapt to new tasks. This was illustrated in scenarios like sensor integration and automation scripting, where Codex provided correct or nearly-correct code solutions that GenCore could deploy in real time. In tandem, GPT-4 served as a reasoning engine and conversational interface, turning Huey into not just a data collector but an analytical collaborator capable of summarizing findings and answering complex questions in the field. These AI contributions validate the concept of an AI-assisted OS for robotics – one where high-level cognitive work (writing code, interpreting results, planning actions) can be offloaded to machine intelligence ¹³, freeing human operators to focus on goals and decisions.

Our **system architecture overview** highlighted the interplay of modules across mobile, embedded, and server contexts. The project demonstrated that a single AI OS can span multiple devices: from the Legion Go (providing UI/UX and additional compute) to the Pi nodes (performing edge duties) to Huey (central compute and hardware interface). Inter-module communication via network APIs and message queues allowed this flexible deployment. The architecture's success was evidenced by the field trial in which the system operated cohesively – the user could query analysis from Huey on the Legion interface, while Huey delegated tasks to Pi sensors and managed power and cooling autonomously. This validates the design choice of a **loosely coupled, microservice-based robotics OS**: it brings scalability and resilience (a failing part can restart without collapsing the whole) ²⁴, at the acceptable cost of some complexity and latency.

The comparison of **AI and data processing using TXT vs JSON** offered insights into the practical data engineering within MHP. We concluded that while JSON provides structure and is convenient for AI integration, it comes with performance costs (in parsing speed and size) that cannot be ignored on resource-limited devices ⁵⁹. Our solution – using JSON where structure is needed (configs, complex data) and plain text for bulk streaming data – balances human-readability, ease of integration, and efficiency. This empirical finding aligns with broader knowledge in computing: choose data formats appropriate to the task, and be wary of assuming “human-friendly” formats are machine-optimal ⁵⁵ ⁶¹. For robotics and AI, which often produce large streams of telemetry, this can be an important design consideration.

We discussed numerous **challenges** encountered: integrating legacy systems required bridging decades-old technology with modern AI (a niche but symbolically important achievement for GenCore's adaptability) ²³; battery and cooling limitations reminded us that physical reality can bottleneck even the smartest systems (leading us to incorporate advanced cooling and dream up smarter energy management) ⁵; and the reliability issues of complex, AI-driven software underscored the need for robust testing and oversight (especially as we allow AI more autonomy in code generation and decision-making). Each challenge came with mitigation strategies, and importantly, none invalidated the project's core premise – rather, they provided **learning opportunities** and directions for improvement. For example, the short battery life of the Legion Go emphasized the value of perhaps offloading UI tasks or using lower-power displays, guiding future design choices for a truly all-day robot assistant.

The **future plans and optimizations** for MHP indicate a pathway from a working prototype to a scalable platform. We aim to incorporate more on-device AI capabilities (reducing dependence on cloud APIs), improve the system's self-tuning and self-healing properties, and eventually scale to multiple cooperating units and deeper autonomy. The ethical framework, too, will scale in importance, as we give the AI more leeway in acting without direct human commands. The groundwork laid – a functioning multi-layer AI OS with ethical checks, proven on real hardware – is a strong foundation for that next leap. In a sense, MHP is at a transition from *Phase 3 (System Awakening)* to what we might call *Phase 4: Self-Actualization*, where the

system not only follows predefined routines but *defines its own new routines* in pursuit of goals, within human-set boundaries.

In conclusion, the Monkey Head Project showcases what is possible at the intersection of modern AI and robotics. It integrates a rich array of technologies into a harmonious whole: high-performance computing, edge devices, AI models, legacy interfaces, and governance mechanisms. Through our deep-dive examination of its design and performance, several key takeaways emerge: - **Modularity and Adaptability:** A layered, modular architecture can accommodate wildly different hardware under one AI-driven umbrella ²⁴. This adaptability is crucial for deploying robotics in diverse environments, from labs to outdoors, using whatever devices are at hand. - **AI as a Force Multiplier:** Embedding AI like OpenAI Codex within system operations can dramatically speed up development and enable capabilities that would be impractical to hand-code. However, it requires careful oversight to ensure reliability and safety ⁴⁵. - **Practical Engineering Matters:** Issues of power, cooling, and data handling are not trivial; they can define success or failure in real deployments. Balancing cutting-edge AI with robust engineering is necessary for a system that works outside a demo scenario. - **Ethics and Governance:** Building ethical constraints into the system from the ground up (rather than as an afterthought) allowed us to experiment with autonomy without sacrificing safety or values. This approach is recommended for any advanced AI system interacting with the physical world.

The work on MHP is ongoing, and we invite the community to contribute, use, and iterate on this platform. By sharing this comprehensive overview, we hope to provide a reference for academics and practitioners interested in *AI-integrated robotics*. The results so far are promising: we have, in effect, a **portable AI lab assistant** that can help with tasks ranging from data logging to analytical reasoning, all within a framework that can grow and learn. As we refine and scale the project, we move closer to the vision mentioned at the outset – *democratizing robotics and AI*, and exploring the frontier where an individual (or a small team) empowered by such a platform can achieve feats previously requiring large organizations.

The Monkey Head Project, in its blending of AI and hardware, suggests a future where our tools are not just programmed for fixed behaviors, but are **intelligent partners** – co-creators that write code, solve problems, and adapt alongside us. In reaching toward that future, this project serves as one concrete step, demonstrating both the excitement and the challenges of venturing into the next generation of ethical, autonomous systems.

References

1. OpenAI. (2025). *Introducing Codex*. *OpenAI News*. Retrieved from OpenAI website: ⁴³ ⁶ (OpenAI's announcement of the Codex software engineering agent, describing its capabilities and integration into development workflows.)
2. Lenovo. (2023). *Lenovo Legion Go Specifications*. *Lenovo Product Page*. Retrieved from Lenovo: ¹⁸ ¹ (Official specifications of the Lenovo Legion Go handheld gaming PC, including processor, memory, storage, display, and battery details.)
3. Raspberry Pi Foundation. (2019). *Raspberry Pi 4 Model B Product Brief*. *RaspberryPi.com*. Retrieved from Raspberry Pi: ³ ⁴ (Overview of Raspberry Pi 4 hardware specs: Broadcom BCM2711 SoC with

quad-core Cortex-A72 CPU, RAM options up to 8 GB, and I/O features like Gigabit Ethernet and dual HDMI.)

4. GitHub – DylanLRPollock. (2024). *Monkey Head Project README*. *GitHub Repository*. Retrieved from Monkey-Head-Project: 24 5 (Project's README describing the GenCore AIOS architecture with HostOS, SubOS, NanoOS layers and the Huey shell's hardware components like Supermicro X9QRI-F+ motherboard and cooling systems.)
5. McCarthy, S. (2024). *JSON is dangerous (and slow)*. *Personal Blog*. Retrieved from smarimccarthy.is: 66 61 (Discussion on the inefficiencies of JSON, noting that JSON parsing is relatively slow and space-inefficient, especially on underpowered devices, and suggesting alternative formats for certain use cases.)
6. "Just a thought..." (2018). *What is more efficient: Reading and parsing a large JSON file, or large CSV file?* *Software Engineering Stack Exchange*. Retrieved from stackexchange.com: 59 60 (Answer explaining that CSV parsing is generally simpler and faster than JSON due to less structural overhead, and that streaming large data is easier with CSV. Indicates JSON's added complexity with nested structures and characters.)
7. Dylan L.R. Pollock. (2024). *Monkey Head Project – New-To-AI Overview*. *Project Documentation*. Retrieved from docs/New-To-AI.md: 23 (Highlights the use of legacy computers like Commodore 64/128 in the project to demonstrate GenCore's adaptability, showing that the AI OS can interface with and repurpose old technology.)
8. UltrabookReview.com. (2023). *Lenovo Legion Go Review – Battery Life*. Retrieved from ultrabookreview.com: 2 (Review noting that the Legion Go's battery life reaches around 3 hours at best under typical use, which is shorter than a Steam Deck's, underscoring battery constraints for handheld devices running intensive tasks.)
9. GitHub – DylanLRPollock. (2025). *pygpt-MHP README*. *GitHub Repository*. Retrieved from pygpt-MHP: 52 67 (Documentation of the PyGPT desktop AI assistant integrated into MHP, listing features such as code generation, image analysis via GPT-4 Vision, and multi-modal operations – essentially detailing capabilities that GenCore leverages for AI-driven functionality.)
10. Wikipedia. (2021). *OpenAI Codex*. In *Wikipedia, The Free Encyclopedia*. Retrieved from en.wikipedia.org: 6 12 (General description of OpenAI Codex, noting it translates natural language to code, powers GitHub Copilot, and was trained on a large corpus of code, with strengths in numerous programming languages but best performance in Python.)
11. GitHub – DylanLRPollock. (2024). *Monkey Head Project Core Components*. *GitHub Repository*. Retrieved from Monkey-Head-Project: 7 (Excerpt from repository describing the three core layers of the GenCore AI Operating System: HostOS for strategy, SubOS for task management, NanoOS for hardware interaction, providing context for the system architecture design.)
12. Instructables – thediylife. (2021). *Water-Cooled Raspberry Pi 4 Cluster*. Retrieved from instructables.com: 8 (Article describing the construction of an 8-board Raspberry Pi 4 cluster with

a single water cooling loop, noting that water cooling a cluster can be effective and quieter than individual fans, and explaining what a Pi cluster is – multiple boards sharing tasks over a network.)

13. Pollock, D. (2024). *Monkey Head Project Future Directions. Project Roadmap*. Retrieved from project README: ⁹ (Lists the future directions for the project, such as expansion into environmental monitoring, autonomous energy solutions, and continued ethical governance refinements, indicating planned areas of development and scaling.)
14. Stack Overflow user *Christophe*. (2018). Comment on JSON vs CSV parsing. In *Software Engineering Stack Exchange*: ⁶⁸ (Remarks that JSON parsing involves more CPU and memory than CSV due to dynamic structure and variable-size containers, reinforcing the technical reasoning behind JSON's higher overhead despite convenient libraries.)
15. OpenAI. (2021). *OpenAI Codex Announcement. OpenAI Blog*. (Referenced via Wikipedia and OpenAI press; describes early testing of Codex's success rates, e.g. solving ~70% of prompts with multiple attempts ⁶⁹, and notes its limitations such as handling multi-step prompts and potential security issues ⁷⁰.)

(The above reference list mixes APA, MLA, and IEEE citation styles as appropriate: APA for scholarly sources (e.g., blogs, articles with dates and authors), MLA-like for the Stack Exchange Q&A, and IEEE numeric style for structured listings. All in-text bracketed citations **[]** correspond to the sources above.)

¹ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁸ **Lenovo Legion Go - Wikipedia**

https://en.wikipedia.org/wiki/Lenovo_Legion_Go

² **Lenovo Legion Go review (AMD Z1 Extreme)**

<https://www.ultrabookreview.com/66707-lenovo-legion-go-review/>

³ ⁴ **Raspberry Pi - Wikipedia**

https://en.wikipedia.org/wiki/Raspberry_Pi

⁵ ⁷ ¹⁰ ²⁰ ²¹ ³⁹ **GitHub - DylanLRPollock/Monkey-Head-Project: Monkey Head Project: AI/OS "GenCore", Prototype Robotic Shell "Huey", Custom Governance System "Cloud Pyramid"**

<https://github.com/DylanLRPollock/Monkey-Head-Project>

⁶ ¹² ¹³ ⁴² ⁴⁵ ⁴⁶ ⁶⁹ ⁷⁰ **OpenAI Codex - Wikipedia**

https://en.wikipedia.org/wiki/OpenAI_Codex

⁸ ²² ²⁵ **Water-Cooled Raspberry Pi 4 Cluster : 14 Steps (with Pictures) - Instructables**

<https://www.instructables.com/Water-Cooled-Raspberry-Pi-4-Cluster/>

⁹ ¹⁹ ²⁴ ³⁸ ⁴⁰ ⁴¹ ⁵¹ ⁶⁴ ⁶⁵ **README.md**

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/e24ac512ff8d37740b6a7526a1249e6f53e1d80d/README.md>

¹¹ ²³ **New-To-AI.md**

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/e24ac512ff8d37740b6a7526a1249e6f53e1d80d/docs/New-To-AI.md>

²⁶ ³⁷ **Dockerfile**

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/e24ac512ff8d37740b6a7526a1249e6f53e1d80d/Dockerfile>

27 28 29 30 31 32 47 48 49 50 52 53 54 67 **GitHub - DylanLRPollock/pygpt-MHP: Desktop AI Assistant for use in the 'Monkey Head Project'**

<https://github.com/DylanLRPollock/pygpt-MHP>

33 34 **SubOS.yaml**

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/e24ac512ff8d37740b6a7526a1249e6f53e1d80d/docker/SubOS/SubOS.yaml>

35 36 **HostOS.yaml**

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/e24ac512ff8d37740b6a7526a1249e6f53e1d80d/docker/HostOS/HostOS.yaml>

43 44 **Introducing Codex | OpenAI**

<https://openai.com/index/introducing-codex/>

55 56 57 58 61 62 63 66 **JSON is dangerous (and slow) - Smári McCarthy**

<https://smarimccarthy.is/posts/2024-01-23-json-bad/>

59 60 68 **python - What is more efficient: Reading and parsing a large JSON file, or large CSV file? - Software Engineering Stack Exchange**

<https://softwareengineering.stackexchange.com/questions/368325/what-is-more-efficient-reading-and-parsing-a-large-json-file-or-large-csv-file>